Sample Chapter: Core JDO ISBN 0-13-140731-7

The chapters of the book titled "Core JDO" are being made available for the purposes of review to allow the community to participate in the review and editing of the chapters of the book.

To add your comments

- Simply send your uninhibited comments and feedback to the authors at jdobook@yahoogroups.com
- Or click and add a note using Acrobat where needed. E-mail the PDF back to the authors at jdobook@yahoogroups.com
- If you would like to receive the chapters in an alternate format, have any questions, concerns or please contact the authors at jdobook@yahoogr oups.com

CHAPTER 6

"If you can find a path with no obstacles, it probably doesn't lead anywhere".
-Frank A. Clark

Finding Your Data

The capability to transparently persist object instances directly is a powerful feature; however it would be incomplete without a mechanism to query and retrieve the information stored in those persistent instances. The JDO specification include a query facility, pieces of which have already been introduced and used in previous chapters. The query facility consists of two parts – the API that can be used in application code to programmatically locate the information and a query language called JDO Query Language (JDOQL) that defines the grammar associated with structuring the queries. This concept is analogous to how traditional the relational database persistence works in Java. There is an API called JDBC that specifies the programmatic contract and the query language (SQL) that defines the grammar. In this chapter we will look at the query API that the JDO provides as a part of the `javax.jdo` package and also look at JDOQL, its syntax and how it can be used for specifying meaningful queries.

6.1 FINDING AN OBJECT BY IDENTITY

In Chapter 1 we introduced the concept of persistence by reachability and how an entire object graph of dependent objects is persisted when an object is saved. The simplest way to find an object is through its identifier. In Chapter 3 (Getting Started) and Chapter 5 (Developing with JDO) we covered the concepts surrounding object identity and how the `getObjectID()` can be used. To recall, the `JDOHelper` class exposes a `getObjectID(Object pc)` method that returns the JDO identity associated with the parameter.

The `PersistenceManager` interface exposes a corresponding `getObjectById()` method can be used in the application code to retrieve the instance through its JDO identity. Here is some example code taken from the example "`ObjectIdentityExample.java`" in Chapter 5 demonstrating how the identity of a persistent object can be used to retrieve it:

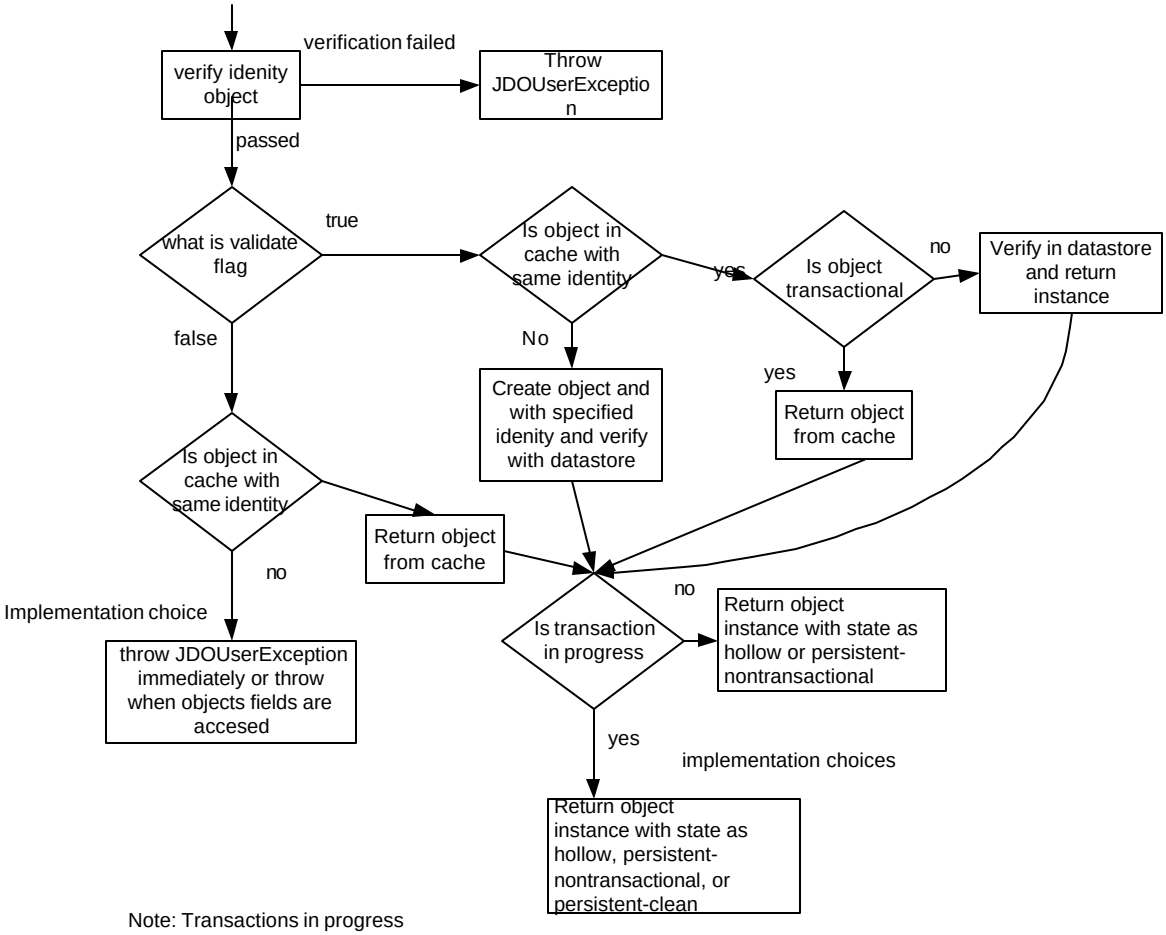
```
PersistenceManager pm = pmf.getPersistenceManager();
Transaction tx = pm.currentTransaction();
tx.begin();

Author author =
  new Author(
    "Sameer Tyagi",
    new Address(
      "1 Main Street", "Boston", "MA", "01822"));
// make the Author and all its fields persistent
pm.makePersistent(author);
tx.commit();
// obtain the object id
Object oid = pm.getObjectId(author);
pm.close();
// conceptually the oid is now available and can be reused

PersistenceManager pm2 = pmf.getPersistenceManager();
Transaction tx2 = pm2.currentTransaction();
tx2.begin();
Author author2 = (Author) pm.getObjectById(oid, true);
// Should print out the name “Sameer Tyagi” below
System.out.println("Author is: " + author2.getName());
tx2.commit();
pm2.close();
```

> The object specifying the identity and returned by `getObjectId()` is required to be serializeable so that applications can store it.

Figure 1 shows the logical flow that the JDO runtime will go through when the `getObjectById()` method is invoked. The JDO implementation tries to locate the object in `PersistenceManagers` cache. If it is found, the object is verified with the underlying datastore and then returned. If the object cannot be found in the cache with the corresponding identity the JDO implementation creates an instance, assigns it the specified identity and returns it. Note that the JDO implementation verifies state with the underlying datastore only

if the object instance in cache is non transactional. Transactional instances do not necessarily need their state to be verified because it is synchronized with the data store.
The semantics associated with retrieving an object instance in JDO by using the JDO identity and the `getObjectById` method are in many ways analogous to the `getHomeHandle()` method in the EJB specifications. EJB can serialize and store the home handle and use it to re-obtain a reference to the remote home object at a later time.

> `getObjectById` returns a hollow instance, which can be deleted afterwards by some other client of the data store. When the application tries to use the object , ie when it is resolved, a `JDOUserException` will be thrown



Note: Transactions in progress
Data store transaction:The returned instance will be marked as persistent-clean.
Optimistic transaction :The re-turned instance will be marked as persistent-nontransactional.

More about states in Chapter 4

Figure 1: Flowchart for `getObjectById()`

6.2 FINDING A SET OF OBJECTS USING AN EXTENT

Earlier in Chapter 3 and Chapter 5 we discussed the `javax.jdo.Extent` interface. The `Extent` interface offers a convenient programmatic way to retrieve a collection of similar objects or an object hierarchy from the underlying datastore.
Recall that an `Extent` shown in Figure 2 represents a collection of all instances in the datastore of a particular persistence capable class. An `Extent` can consist of just instances of the single class or can additionally include instances of subclasses too. By default it is possible to get an `Extent` for any persistence capable class.
An application obtain an `Extent` for a class from a `PersistenceManager` and then use a the `iterator()` method to go through all instances using a `java.util.Iterator`

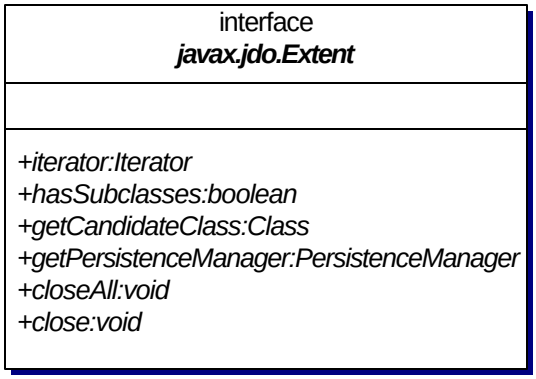


Figure 2: The `javax.jdo.Extent` interface

The following code snippet shown again from `ReadByExtentExample.java` in Chapter 3 demonstrates the use of an `Extent` in finding instances of a particular class.

```
tx.begin();
Extent extent = pm.getExtent(Author.class, true);
Iterator results = extent.iterator();
while (results.hasNext()) {
    Author author = (Author) results.next();
    String name = author.getName();
    System.out.println("Author's name is '" + name + "'.");
    }
extent.closeAll();
tx.commit();
```

Note that when used alone in the manner shown above, the `Iterator` can potentially include a lot of objects since it returns the complete hierarchy-ie all the instances of Author and the subclasses

The `Extent interface` offers a convenient mechanism to locate persistent instances of a particular persistence capable class in the data store however most application would need a more specific mechanism to locate instances that match specific criteria.

6.3 FINDING OBJECTS WITH THE QUERY FACILITY

Obtaining specific objects by using their JDO Identity or obtaining a collection of objects through the use of an `Extent` is simple and convenient. However both these techniques have their drawbacks. It is not always possible or desirable to

- Store the JDO identity of an object
- Iterate through potentially large collections of object graphs to locate a few objects that might be useful to the application. For example if the previous snippet was applied to `Stock` objects that had a `symbol` attribute, the application would need to iterate through all the `Stocks` and compare the `symbol` until it found the one it was interested in !

The API portion of the query facility in JDO consists primarily of the `Query` interface shown in Figure 3. This was also introduced in Chapter 3 and Chapter 5 earlier.

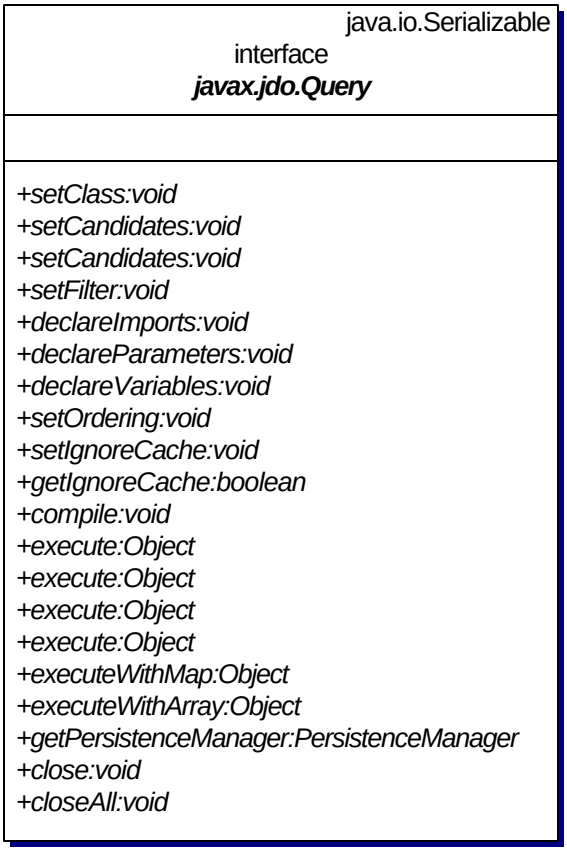


Figure 3: The `javax.jdo.Query` interface

The `Query` interface abstracts the concept of interrogating the underlying data store and retrieving *objects* that might match specific *criteria*. There are three terms that are important in this regard

1. The candidate class: This corresponds to the highest level in the class inheritance hierarchy that the results must belong too. This can be conceptually thought of as the branch where the JDO implementation will *prune* the class inheritance tree. For example in Figure 4 if the application was only interested in finding specific Employees, then the candidate class would be `Employee` rather than `People`. If the application was interested in finding gold customers, then the class would be `Gold`, rather than `Customers` or `People`.

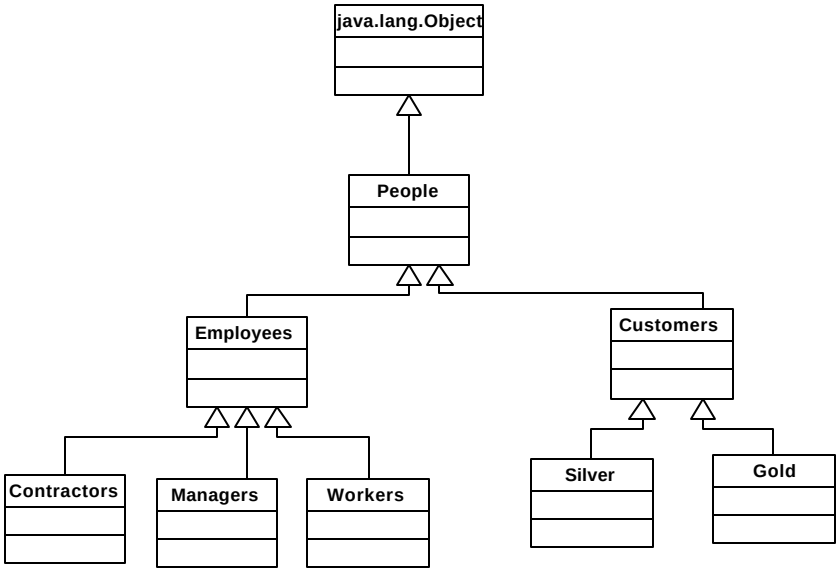


Figure 4: Deciding on the candidate class.

2. The candidate collection: The candidate collection is a `Collection` or `Extent` of objects from a corresponding candidate class that are present in the data store. An application can pass a candidate collection that it constructs to the query facility and have that collection narrowed or filtered as a result of the query execution.
3. The query filter: This is an expression that evaluates to some `boolean` condition and is used to narrow the candidate collection returned as a result of the query execution.

Let us rework the previous example to incorporate the `Query` facility. Note that the results produced by the code shown below will be identical to the earlier example. It will return all persistent `Author` objects since no `Extent` or *filter* has been specified in the query.

```
PersistenceManager pm = pmf.getPersistenceManager();
Transaction tx = pm.currentTransaction();
tx.begin();
     Query query = pm.newQuery(Author.class);
     Collection results = (Collection) query.execute();
     if (results.isEmpty())
```

```
    System.out.println("No results found");
else {
  Iterator iter = results.iterator();
  while (iter.hasNext()) {
    Author author = (Author) iter.next();
    String name = author.getName();
    System.out.println("Author's name is " + name);
    }
  }
query.closeAll();
tx.commit();
```

When the `Query` is executed via the `execute()` method, the results are contained in an un-modifiable or immutable `Collection` .The actual method signature is declared to return an `Object` to allow for future or vendor extensions but the application code must explicitly cast the result of the `execute()` method to a `Collection`. Elements of the collection can be accessed through the `Iterator` but any attempt to change the collection will cause an `UnsupportedOperationException` to be thrown.

The `Collection` object containing as the query results is only required to support the `iterator()` method because the implementation of the class is up to the vendor. For example a vendor using a relational database may not be able to support the `size()` method because the database rows may only be available the first time they are accessed. For this reason the `size()` method may return `Integer.MAX_VALUE.`

The results of a query may hold on to resources linked to the underlying data store (like connections). Therefore all the query instances must be closed explicitly when the results are no longer needed. The individual result can be closed using the `close(Object queryResult)` method or all open results associated with the `Query` can be closed at once with the `closeAll()` method. The idea of closing `Query` results is conceptually analogous to the way the `close()` method works for the `java.sql.ResultSet` object in JDBC.

6.3.1 Queries with an Extent

Though the `Extent` by itself may not be too meaningful, when used alongside the `Query` it can be used to refine the results. The `Query` can be refined to return only instances of a class and its subclasses by specifying an `Extent` when the `Query` is constructed using the factory method. This is shown below

```
tx.begin();
```

```
Extent extent = pm.getExtent(Author.class, true);
Query query= pm.newQuery(extent);
Collection results=(Collection)query.execute();
Iterator iter = results.iterator();
while (iter.hasNext()) {
    Author author = (Author) iter.next();
    String name = author.getName();
    System.out.println("Author's name is '" + name + "'.");
    }
query.closeAll();
tx.commit();
```

Again the results produced by the above code will be identical to those produced in section 6.2 since the `Extent` specifies the `Author` class.

The preceding discussion was meant to introduce you to the API itself. Before we look at queries and the API in greater detail let us switch tracks and look at the second part of the query facility, JDOQL -the query language. We will return to the API and look at examples in greater detail and by incorporating the query language in them.

6.4 JDOQL

6.4.1 WHAT IS JDOQL?

The filter is essentially a string passed to the query facility that is used by the query facility to refine the results returned. The syntax for the filter is specified using JDOQL or the JDO Query language. This begs the question. Why a new syntax, why not use Structured Query Language (SQL) that is standard to all relational databases? One of the goals of JDO is to isolate the developer from the specifics of the underlying datastore. This means that the query language must be vendor and implementation agnostic. The vendor is free to choose SQL, Object Query Language (OQL), or whatever other query language the underlying datastore supports. To quote the specifications, the primary design goals for the query language were:

- "Query language neutrality: The underlying query language might be a relational query language such as SQL; an object database query language such as OQL; or a specialized API to a hierarchical database or mainframe EIS system.

- Optimization to specific query language: The Query interface is capable of optimizations; therefore, the interface has enough user-specified information to allow for the JDO implementation to exploit data source specific query features.
- Accommodation of multi-tier architectures: Queries may be executed entirely in memory, or may be delegated to a back end query engine. The JDO Query interface provides for both types of query execution strategies.
- Large result set support: Queries might return massive numbers of JDO instances that match the query. The JDO Query architecture provides for processing the results within the resource constraints of the execution environment.
- Compiled query support: Parsing queries may be resource-intensive, and in many applications can be done during application development or deployment, prior to execution time. The query interface allows for compiling queries and binding runtime parameters to the bound queries for execution."

The other goal was to keep the query facility simple and developer friendly. JDOQL achieves this by using the standard Java operators and syntax that developers are already familiar with. This feature has been acclaimed and criticized equally. On one hand, JDOQL is easy to construct and program, but on the other hand most developers are already familiar with SQL and database operators are capable of producing complex, highly optimized, SQL queries. For many architects embedding queries in a middle tier transparent persistence API like JDO is an unwelcome step. However this has more to do with the overall use of JDO in an enterprise than the query facility. As per its design goal JDO needs to isolate the application from the underlying datastore. Using SQL would mean tying the implementation to some sort of relational implementation. However it should be noted that JDOQL queries can only return JDO instances. There are no projections or simple type/value queries, or even COUNT queries, like in SQL.

Using a query language other than JDOQL may make the application non portable across vendor implementations.

JDOQL is one of many languages that an implementation can support and all implementations are required to support the specification defined behavior of JDOQL. This keeps the application code portable. In reality vendor implementations that use relational databases as the underlying store allow the use of SQL directly as we will see in section 6.6.4.

A query can be constructed for other vendor supported languages using the `newQuery (String language, Object query)` method in the `PersistenceManager`.

The value of the string to specify JDOQL is "`javax.jdo.query.JDOQL`". For other languages it is a vendor specific string.

6.4.2 How is a something like JDOQL specified?

The syntax for specifying syntax, i.e. the meta-syntax is done using a standard notion like Backus Normal Form (BNF). BNF is a widely used and accepted, notation for specifying the syntax of programming languages like Java and C++ and query languages like SQL, JDOQL etc. Fortunately the only people who need to know BNF are the specification authors! Developers only need to know how JDQL works. Interested readers can refer to Appendix C for the BNF notation of JDOQL.

6.4.3 Specifying Filters

The filter string defined in JDQL must evaluate to a `boolean` condition. The string itself can be composed of multiple expressions that are separated using the standard Java syntax for `AND`, `OR` and `NOT` operators. The logical operators are explained further in Table 1.Each of the expressions can use the standard Java operators used for comparison. These are explained further in Table 2.

The `boolean` expression specified through the filter can be used to

- Specify the criteria for the query
- Determine if an instance in the candidate collection is a member of the result set that is specified by the query.

For example, the following code segment from `SimpleQueryWithFilter.java` shows how a query can be constructed to find all `Author` objects where the authors name is "Snoop Smith".

```
tx.begin();
  // find all author objects where the name is Snoop Smith
String filter = "name==\"Snoop Smith\" ";
Query query = pm.newQuery(Author.class, filter);
Collection results = (Collection) query.execute();
if (results.isEmpty())
     System.out.println("No results found");
else {
    Iterator iter = results.iterator();
```

```
      while (iter.hasNext()) {
      Author author = (Author) iter.next();
      String name = author.getName();
      System.out.println("Author's name is " + name);
      System.out.println("Author's address is " +
                           author.getAddress());
      System.out.println("Author's books are " );
      Iterator books=author.getBooks().iterator();
      while(books.hasNext())
          System.out.println("Book:" +books.next()) ;
           }
        }
query.closeAll();
tx.commit();
```

The filter and candidate collection can be specified when the query is constructed through one of the `newQuery()` methods or dynamically using the `setFilter()` and `setCandidates()` methods respectively.
The `setCandidates()` method binds the candidate collection to the query instance. If the collection is modified after this invocation it is up to the vendor implementation to allow the modified elements to participate in the query or throw a `NoSuchElementException` during execution of the `Query` for those elements.

The elements in the candidate `Collection` must be persistent instances associated with the same `PersistenceManager` as the `Query` instance. Vendors can optionally choose to support transient instances in the collection but relying on that feature would make the code non portable to other vendor implementations. If there are multiple `PersistenceManagers` and any element in the collection is associated with a persistence manager other than the one under which the query is constructed, a `JDOUserException` will be thrown when the query is executed.

The code segment below from `SimpleQueryWithFilterAndCandidates.java` reworks the previous example to use a candidate collection and filter.

```
          tx.begin();
          Author a1= new Author("Author Smith");
          Author a2= new Author("Author Jones");
          Author a3= new Author("Author Knowles");
          Collection candidates = new ArrayList();
          candidates.add(a1);
          candidates.add(a2);
          candidates.add(a3);
```

```
        pm.makePersistentAll(candidates);

    // find all author objects where the name is Author Smith
        String filter = "name==\"Author Smith\" ";
        Query query = pm.newQuery(Author.class, filter);
        query.setCandidates(candidates);
        Collection results = (Collection) query.execute();
```

The preceding example makes the candidate collection persistent. This is important for the code above to work since the candidate collection must represent objects in the data store. If the vendor doesn't support transient instances in the candidate collection then the code segment below will throw a `JDOUserException`.

```
        Author a1= new Author("Author Smith");
        Author a2= new Author("Author Jones");
        Author a3= new Author("Author Knowles");
        Collection candidates = new ArrayList();
        candidates.add(a1);
        candidates.add(a2);
        candidates.add(a3);
    // find all author objects where the name is Author Smith
        String filter = "name==\"Author Smith\" ";
        Query query = pm.newQuery(Author.class, filter);
        query.setCandidates(candidates);
```

> Support for transient instances in the collection is optional and vendors may, may not implement it.

Operator	Description	Supported data types	Example	SQL Equivalent
!	Logical compliment	Boolean, boolean	`String filter = "!(employed==true)"`	NOT
&&	Conditional AND	Boolean, boolean	`String filter = "(name==\"John\") && (age==30)"`	AND
||	Conditional OR	Boolean, boolean	`String filter = "(name==\"John\")|| (name==\"Johnathan\")"`	OR

Table 1: Logical operators

JDOQL filters cannot change any fields. They are non-mutating. Therefore the Java operators "&" and "| "are should not be used.

Operator	Description	Supported data types	Example	SQL Equivalent
==	Equals	byte, short, int, long, char, Byte, Short, Integer, Long, Character float, double, Float, Double, BigDecimal, BigInteger, Boolean, boolean, Date, any class instance or array	Name equals John:= String filter = "(name==\"John\")"	==
!=	Not equal	byte, short, int, long, char, Byte, Short, Integer, Long, Character float, double, Float, Double, BigDecimal, BigInteger Boolean, boolean, Date, any class instance or array	Name does not equal John:= String filter =" (name!=\"John\") "	<>
>	Greater than	byte, short, int, long, char, Byte, Short, Integer, Long, Character float, double, Float, Double, BigDecimal, BigInteger, Date, String	Age is greater than 30:= String filter = "age > 30"	>
<	Less than	byte, short, int, long, char, Byte, Short, Integer, Long, Character float, double, Float, Double, BigDecimal, BigInteger, Date, String	Age is less than 30:= String filter = "age<30"	<
>=	Greater than or equal	byte, short, int, long, char, Byte, Short, Integer, Long, Character float, double, Float, Double, BigDecimal,	Age is greater than or equal to 30:= String filter = "age >= 30"	>=

		BigInteger, Date, String		
<=	Less than or equal	byte, short, int, long, char, Byte, Short, Integer, Long, Character float, double, Float, Double, BigDecimal, BigInteger, Date, String	Age is less than or equal to 30:= String filter = “age<=30”	<=
+	Binary addition	byte, short, int, long, char, Byte, Short, Integer, Long, Character float, double, Float, Double, BigDecimal, BigInteger	The persons age is 2 yrs below the minimum age String filter =”age+2>=minage”	+
-	Binary subtrac-tion	byte, short, int, long, char, Byte, Short, Integer, Long, Character float, double, Float, Double, BigDecimal, BigInteger	The persons age is 2 years more than the minimum age String filter =”age-2>minage”	-
~	Negation (operator is followed by a primi-tive)	byte, short, int, long, char, Byte, Short, Integer, Long, Character	Assume field x is nega-tive, change to positive value String filter=”age==(~x)”	(-expr) - 1
*	Multiplica-tion	byte, short, int, long, char, Byte, Short, Integer, Long, Character float, double, Float, Double, BigDecimal, BigInteger	Profile is 2% of price String filter=”.02 * price”	*
/	Division	byte, short, int, long, char, Byte, Short, Integer, Long, Character float, double, Float, Double, BigDecimal, BigInteger	Profile is 2% of price String filter=”(2/100) * price”	/

Table 2: Comparison operators

Operators relating to references

Operator	Description	Example
`()`	Used to cast object types.	Author names start with 'S": `String filter = "((String)name).startsWith(\"S\") ";` Note that the cast in the above filter is not explicitly required but is only meant to show usage.

`.` (period)	Used to reference fields as in the Java language	Author names start with 'S" :
		`String filter="name.startsWith(\"S\")"`
		Author living in zip code 02929:
		`String filter = "address.zipcode==\"02929\"";`

`this`	Used to access the hidden fields. `this` refers to an object of the candidate class.	`String filer= "this.name == name";` More details on `this` in Section 6.5.5

Table 3: Special operators

6.4.3.1 DIFFERENCES BETWEEN JAVA AND JDOQL OPERATORS

Though the JDOQL operators follow the syntax of the Java operators, there are some subtle and explicit differences between the operators as used in the query filters and as used in the Java language.

6.4.3.1.1 Equality and Ordering

- In Java the equality operator "==" cannot be used between a primitive and its corresponding wrapper. JDOQL allows this operator to be used between primitives and their corresponding wrapper classes. For example (`10==objectten`) is a valid use where `objectten` is an instance of `Integer.`
- The same concept extends to the ordering operators (`>, <, >=, <=`).In Java one cannot use these operators between a primitive and a wrapper. For example the syntax (`10>objectten` would be illegal in Java if the variable `objectten` referred to an instance of `Integer` and not a primitive. Like the equality operator, JDOQL allows for this type of usage between primitives and wrappers.

- The use of the equality and ordering operators extends to the `String` and `Date` objects. For example, if d1 and d2 represent two `Date` instances, `(d1>d2)` would be illegal in Java but is valid as a part of the filter in JDOQL.

6.4.3.1.2 Assignment operators

The query filter is not allowed to change the value of a persistent field. This means that the assignment operators `=, /=,+=,` etc. and pre/ post increment/decrement are not allowed. For example the following string though legal in Java would be illegal as a filter in JDO.

```
String filter="author.name=\"Snoop Smith\"";
```

6.4.3.1.3 Methods

For performance reasons, a JDO implementation is not required to instantiate an object in order to evaluate the query filter. Additionally the query may execute on a server where the application is not available. This means that the method invocation as a part of the filter is not supported and considered illegal. For example the following filter is illegal:

```
String filter="author.getName()=="Snoop Smith";
```

the only exception when it comes to methods is the `Collection` interface and the String class. There are two methods that can be invoked on a `Collection` instance in a filter:

- `isEmpty()`: This will return `true` if the collection contains no elements or if the reference is null.
- `contains(Object obj)`: This will return `true` if the collection contains the object passed as the parameter. Note that the reference passed must be a persistent capable object. The JDO implementation will use the JDO identity, as opposed to the `equals()` method to check if the object exists in the collection.

There are two methods can be invoked on a `String` instance in a filter:

- `startsWith(String str)`: If the persistence field (a `String` instance) starts with the specified parameter then the method returns true. For example the name is a persietent field in the `Author` class. A valid filter could be `String filter="name.startsWith(\"S\")"`
- `endsWith(String str)`: This is similar to the `startsWith()` method other than it checks if the field ends with the specified parameter.

6.4.3.1.3 Navigation

The term navigation in JDOQL is used to describe to the ability to explicitly traverse and resolve references to fields in a persistence capable class. As shown in Table 3, this is done using the period (`"."`) operator.

- JDOQL requires that navigation through a field that has a null value results in the sub expression being evaluated to false. In the above example if the `zipcode` field in the candidate class was null, it would result in the filter evaluating to `false`. For example the filter to determine if an author resides in the zip 02929 would look like:-
  ```
  String filter = "address.zipcode==\"02929\"";
  ```
  In this case the filter *navigates* to the field `zipcode` via the address reference.
- JDOQL also supports navigation through a `Collection` types) using variables and the `contains()` method. This is covered further in section 6.5.3.

6.5 QUERIES, FILTERS AND OPTIONAL PARAMETERS

6.5.1 DECLARING PARAMETERS

Rather than hard coding the values for certain objects in the filter string, the filter can contain placeholders that can be replaced with different values. This allows a single query to be executed multiple times with new values for the placeholder. For repeated queries, vendors can optimize their underlying implementation specific to the data store for optimizing performance.

Conceptually the usage of parameters in JDOQL is similar to the way parameters are passed in JDBC prepared statements. For example in JDBC the hard coded SQL would look like:

```
Statetment stmt= con.createStatement("UPDATE AUTHORS SET
BOOK= 'Core JDO" WHERE NAME=='Snoop Smith'");
```

This can be replaced with a `PreparedStatment` as

```
PreparedStatement stmt = con.prepareStatement(
  "UPDATE AUTHORS SET BOOK =? WHERE NAME ==?");
```

The prepared statement can be executed repeatedly with new values replacing the `"?"` through the `stmt.setXXX()` methods .The database pre-compiles the SQL and executes faster.

JDOQL works in a similar manner. Rather than containing "?" in the filter string, actual variable names may be used. The values for the variables are passed using the `declareParameters()` method in the `Query` interface which takes a String as an argument. The syntax of the string is the same as that used in standard Java method signatures i.e. it is a type declaration followed by the variable name. Let us look at an example of how

this works. The code segment below from `FilterExamples.java` shows how to find all books with a given publisher name and a publication date earlier than the current date. The same query is then re-executed with new values for the publisher's name.

```
    // find all books with a certain publisher name and date
// First create the filter
  String filter = "publisher.name== pubname &&
                                (publicationDate < today";
  Query query = pm.newQuery(Book.class, filter);
  Date today= new Date();
  String pubname="Pet publishin Co";
// Declare any import statements (see next paragraph for
// explaination)
  query.declareImports("import java.util.Date;")  ;

// Declare the parameters using the standard Java method
// signature syntax
  query.declareParameters("Publisher pubname,Date today");
// execute the query with the given parameters
  results = (Collection) query.execute(pubname,today);
  if (results.isEmpty())
       System.out.println("No results found");
  else {
      Iterator iter = results.iterator();
       while (iter.hasNext()) {
            Book book = (Book) iter.next();
             System.out.println("Book found "+book);
             }
        }

  // repeat query execution with new value  for publisher
   pubname="Books123.com";
   results = (Collection) query.execute(pubname,today);
   if (results.isEmpty())
         System.out.println("No results found");
   else {
        Iterator iter = results.iterator();
        while (iter.hasNext()) {
            Book book = (Book) iter.next();
            System.out.println("Book found "+book);
             }
```

```
}
```

The `Query` interface has four convenient methods for execution that can use parameters. They take one, two or three objetcs as parameters. Usually one of these overloaded methods will suffice. However, if the query takes a number of parameters then the more generic form which takes an array of objects can be used. These methods are described in Table 4

Query execute method	Description
`execute(Object p1)`	Execute the query and return the filtered `Collection`.
`execute(Object p1, Object p2)`	Execute the query and return the filtered `Collection`.
`execute(Object p1,Object p2, Object p3)`	Execute the query and return the filtered `Collection`.
`executeWithArray(Object[] parameters)`	Execute the query and return the filtered `Collection`

Table 4: Query execution methods

6.5.2 DECLARING IMPORTS

Sometimes the class or interface definitions of parameters passed to the query may reside in a package other than the candidate class. These definitions can be imported on demand by the JDO implementation using the `declareImports()` method. The method takes a semicolon separated list of import statements in the standard Java syntax. In the preceding example the `publicationDate` was used as a parameter but the `Date` class resides in the `java.util` package. The `declareImports()` method was used to pass the import declaration to tell the JDO runtime where to locate the parameter definition. The standard `java.lang.*` classes, the candidate class and classes in the same package as the candidate class do not need to be explicitly imported. All imported classes must however be available in the class path.

6.5.3 DECLARING VARIABLES

JDOQL uses the concept of variables to test if some items in a multi-valued collection match specific criteria. If the filter involves iteration through a `Collection`, a variable can be used along with the `Collection.contains()` method to check if a specific element in the collection matches the filter criteria. For example the `Author` class contains a field named `books` that is of type `java.util.Set`. To check if the `Set` (a sub interface of `java.util.Collection`) contains any Book object with the `title` “Learn Java Today”, the code would look like:

```
String filter = "books.contains(myfavouritebook) && " +
          "(myfavouritebook.title==\"Learn Java Today\")";
Query query = pm.newQuery(Author.class, filter);
query.declareVariables("Book myfavouritebook");
Collection results = (Collection) query.execute();
```

The `contains()` method is passed a variable named `myfavouritebook`. This variable is then declared using the `declareVariables()` method, following the standard Java syntax for variable declarations. (`ie variableType variableName`)

In the preceding example the `Collection` is iterated and each element is set to the variable `myfavouritebook`. The condition `(myfavouritebook.title==\"Learn Java Today\")` is then evaluated for each value and the result formed. Of course this serial access is only conceptual and the JDO implementation may use a strategy specific to the indexing features of the underlying data store. For example in the preceeding code segment a vendor implementation may generate SQL for a relational data store that looks like:

```
Select Distnct T0.Jdoid T0.Address, T0.Name From Author T0,
Author_Books T1, Book T2 Where (T2.Title = ? And T0.Jdoid =
T1.Jdoid And T1.Books = T2.Jdoid)
```

> When declaring variables, the names used should be unique and cannot conflict with other parameter names. Also variables must be specified in a single call.- For example, if multiple variables need to be passed in a filter then all of them should be declared in a single `declareVariables()` call

6.5.4 ORDERING RESULTS

Sometimes it is desirable to have the results of a query ordered in a particular manner. For example `Author` objects ordered by name in ascending alphabetical order. The ordering criteria for a query can be specified as a comma-separated list of field names followed by the keywords `"ascending"` or `"descending"`. The results returned will be ordered using the left to right evaluation of the ordering criteria. The declarations are passed to the `Query` interface using `setOrdering()` method. For example the code segment below from `FilterExamples.java` shows how a query can be executed to return all `Books` sorted in ascending order by `title`. If two books have the same `title` then they will be sorted by the next criteria, descending order of the `publicationDate`.

```
query = pm.newQuery(Book.class);
query.setOrdering("title ascending,
                           publicationDate decending");
results = (Collection) query.execute();
```

6.5.5 NAMESPACES IN A QUERY

A namespace refers to a logical separation of type names, fields and variables. A `Query` has two namespaces:

- The namespace of types and classes.
- The namespace of fields, variables and parameters.

The `setClass()` method adds the name of the candidate class, and the `declareImports()` method adds the names of imported classes or interfaces to the type namespace.

When the `setClass()` method is used to set the candidate class explicitly the names of fields of the candidate class will also be added to namespace of the fields and variables for the `Query`. The same will happen when the `declareParameters()` or `declareVariables()` methods are invoked. I.e. the names of the parameters and variables to are added to the same namespace. A parameter name or variable name *overrules and hides* the field name set by the candidate class if they are the same. The hidden field may be accessed using the `'this'` operator shown in Table 3. The `“this”` keyword in a filter always refers to an object of the candidate class. For example consider the code snippet from `FilterExamples.java`. The filter looks for an `Author` with the `name` field equals “Steven Jones” and declares a variable of type `Book` also called `“name”`

```
String filter = "books.contains(name) && " +
             "(this.name==\"Steven Jones\")";
Query query = pm.newQuery(Author.class, filter);
query.declareVariables("Book name");
Collection results = (Collection) query.execute();
```

In this case the persistent field `“name”` of the candidate class (`Author`) will be *hidden* by the declared variable `“name”` and the `“this”` keyword must be used to refer to it explicitly.

> The `setClass()` method needs to be invoked only if the candidate class was not passed to the `newQuery()` factory method.

6.6 MORE ON THE QUERY INTERFACE

6.6.1 Query Construction

There are a number of factory methods in the `PersistenceManager` interface that can be used to construct a Query instance. We have seen some of these methods in preceding code snippets. Table 5 lists all the factory methods. The methods dealing with query compilation and creating queries from other queries and are covered further in section 6.6.4 and 6.6.5 respectively.

Factory method	Description
`Query newQuery()`	Creates a new `Query` with no elements.
`Query newQuery(Class class)`	Creates a new `Query` specifying the `Class` of the candidate instances.
`Query newQuery(Class cls, Collection candaidates)`	Creates a new `Query` with the candidate `Extent`; the class is taken from the `Extent`.
`Query newQuery(Class class, Collection candaidates, String filter)`	Creates a new `Query` with the `Class` of the candidate instances, candidate `Collection`, and filter.
`Query newQuery(Class class, String filter)`	Creates a new `Query` with the `Class` of the candidate instances and filter.
`Query newQuery(Extent extent)`	Creates a new `Query` with the `Class` of the candidate instances and candidate `Extent`.
`Query newQuery(Extent extent, String filter)`	Creates a new `Query` with the candidate `Extent` and filter; the class is taken from the `Extent`.
`Query newQuery(Object compiled)`	Creates a new `Query` using elements from another `Query`.
`Query newQuery(String language, Object query)`	Creates a new `Query` using the specified language.

Table 5: Factory methods for creating Queries

6.6.2 Queries and the cache

In Chapter 5 the concept of the cache maintained by the `PersistenceManager` was introduced. The Query interface exposes two methods that specify the runtime behavior and query execution characteristics. The `setIgnoreCache(boolean ignoreCache)` and `getIgnoreCache() set and get the javax.jdo.option.IgnoreCache` property. When this property is set to `true`, the query execution will ignore the instances in the `PersistenceManagers` cache. These cached instances may have changed from their persistent representations in the data store (see Chapter 4 and Appendix A for state changes). While synchronizing state for the query execution may result in more accurate results (this may also depend on the isolation settings in the data store) it will also negatively affect the performance characteristics of the system. When the cache is ignored, the query will execute entirely in the data store. This feature when used in conjunction with optimistic transactions or read only type transactions (See Chapter 10 for transactions) can dramatically improve performance.

6.6.3 Compiled Queries

JDO queries can be compiled with the `compile()` method shown in Figure 3. This does not mean that the query is compiled into some native format; it is merely a hint to the implementation to optimize an execution plan for the query. It is up to the vendor to support this optional feature and choose the compilation strategy depending on the data tore. If the vendor chooses a compile time optimization technique a potential disadvantage could be that execution strategy for the query actually may become sub optimal at runtime due to updates to the data store at runtime.

> The notion of compiled queries is analogous to the concept of compiled queries in the object oriented databases which use OQL. OQL is an object -oriented SQL-like query language and is a part of the ODMG-93 standard. It can be used in two different ways either as an embedded function in a programming language or as an ad hoc query language. Some vendors compile OQL declarations into native constructs like C++ to perform query optimization at preprocessing time, rather than run-time. Therefore, all type errors are caught at compile time. (Note that precompiled queries are also available in relational databases)

6.6.4 Template Queries

The vendor provided implementation class for the `Query` interface is required to be `Serializable`. This means that the application code can serialize and save the `Query` instances indefinitely and recreate them subsequently. Since a Query may hold on to active resources in the underlying data tore the de-serialized `Query` object cannot actually be re-executed. However the details associated with the `Query` (e.g. the candidate class, filter string, imports, parameters, variables, and ordering) are retained. This de-serialized query can be used as a sort of template to re-create another query instance from the `PersistenceManager` with the original details that were saved. For example, the code snippet from `TemplateExample.java` shows how the `Query` can be serialized and reused.

```
// create and serialize the query
  if(args[0].equalsIgnoreCase("serialize")) {
    String filter = "books.contains(myfavouritebook) && " +
            "(myfavouritebook.title==\"Learn Java Today\")";
    Query query = pm.newQuery(Author.class, filter);
    query.declareVariables("Book myfavouritebook");
   // close resources just to be safe
     query.closeAll();
    serializeQuery(query);
         }
 // other code here

 /**Method to serialize and save the query object*/
 public static void serializeQuery(Query query){
  try{
    ObjectOutputStream os = new ObjectOutputStream (new
                            FileOutputStream("query.ser"));
    os.writeObject(query);
    os.close();
    } catch(Exception e){
      System.out.println("An exception occurred during query
                        serialization :"+e);
        }

     }
```

The serialized query can then be read later and used as a template and executed. It is required that the `Query` instance being reused must come from the same JDO vendor, in other words one should not expect to serialize and reuse the `Query` instances across vendor implementa-

tions. The corresponding snippet below from the same example shows how the serialized query can be read and used as a template.

```
if (args[0].equalsIgnoreCase("deserialize")) {
// deserialize and execute
Transaction tx = pm.currentTransaction();
tx.begin();
Object deserQuery = deserializeQuery();
Query recreatedQuery = pm.newQuery(deserQuery);
Collection results = (Collection) recreatedQuery.execute();
if (results.isEmpty())
    System.out.println("No results found");
else {
    Iterator iter = results.iterator();
    while (iter.hasNext()) {
    Author author = (Author) iter.next();
    String name = author.getName();
    System.out.println("Author's name is " + name);
    System.out.println("Author's address is " +
                          author.getAddress());
    System.out.println("Author's books are ");
    Iterator books = author.getBooks().iterator();
    while (books.hasNext())
        System.out.println("Book:" + books.next());
          }
      }
  recreatedQuery.closeAll();
  tx.commit();
        }
// other code here

/** Method to Deserialize the query */
public static Query deserializeQuery() {
 try {
    ObjectInputStream os = new ObjectInputStream(new
                        FileInputStream("query.ser"));
    Query q = (Query) os.readObject();
    os.close();
    return q;
     } catch (Exception e) {
       System.out.println("An exception occurred during
                          query de-serialization :" + e);
        return null;
```

```
}
```

6.6.5 Choosing a different query language

The JDOQL discussed in this chapter is required to be implemented by all vendors. However vendors can choose to support other query languages natively in their implementations. For example a vendor using a relational database as the underlying data store may choose to allow SQL as the query language, another vendor using an object database may support OQL directly. A Query can be constructed in a format other than JDOQL using the overloaded `newQuery()` method in the persistence manager.

```
public Query newQuery(String language, Object query);
```

If the vendor supports this feature, the application may potentially leverage features that are specific to the data store, however using anything other than JDOQL may render the application code non-portable to other vendor implementations. The code example below shows how a SQL query an be used in a particular vendors implementation:

```
String SQL= "Select * from Authors";
Query query = pm.newQuery(VendorInterface.SQL, SQL);
Collection results = (Collection)query.execute();
```

Note that the value of the `String` parameter `"language"` is not specified in the JDO specifications for any language, it depends on the vendor.

SUMMARY

In this chapter we looked at how the persistent instances can be queried and retrieved from the underlying data store using the query facilities specified in JDO. We looked at two parts of the facility, the API and JDOQL which is a Java like syntax for specifying the query filters. We looked at detailed examples that showed different features of the API and JDOQL that are available to developers.

In the next chapter we will look at Architecture Scenarios and how JDO fits into different environments.